Data Structures and Algorithms
Lecture notes: Binary Search Trees, Cormen chapter 12

Lecturer: Michel Toulouse

Hanoi University of Science & Technology
michel.toulouse@soict.hust.edu.vn

31 mai 2021

Outline

Search in a link list

Input : A link list L of n nodes and the target item x.

Algorithm : Search starts with the pointer to the head node and continues until either x is found - or the entire link list is searched (next pointer = NULL).

Worst case running time is $O(n)$

Binary Search Trees

A set of elementary data structures (nodes) link together by pointers such to form a binary tree data structure as a whole.

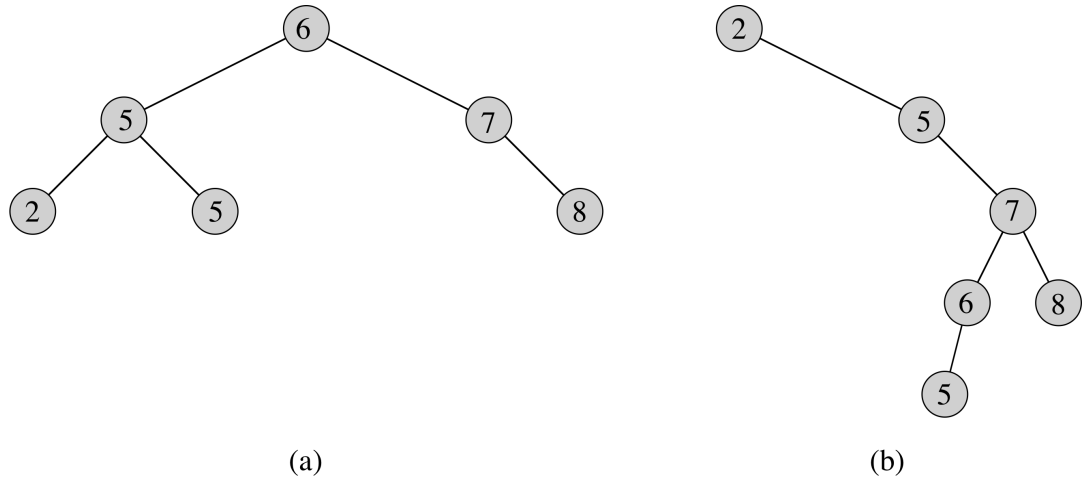
Each node has :

- *key* (data) : an identifying field inducing a total ordering
- *left* : pointer to a left child (may be NULL)
- *right* : pointer to a right child (may be NULL)
- *p* : pointer to a parent node (NULL for root)

Binary Search Trees

Binary search tree should satisfy the following property :
$x.left.key \leq x.key \leq x.right.key$

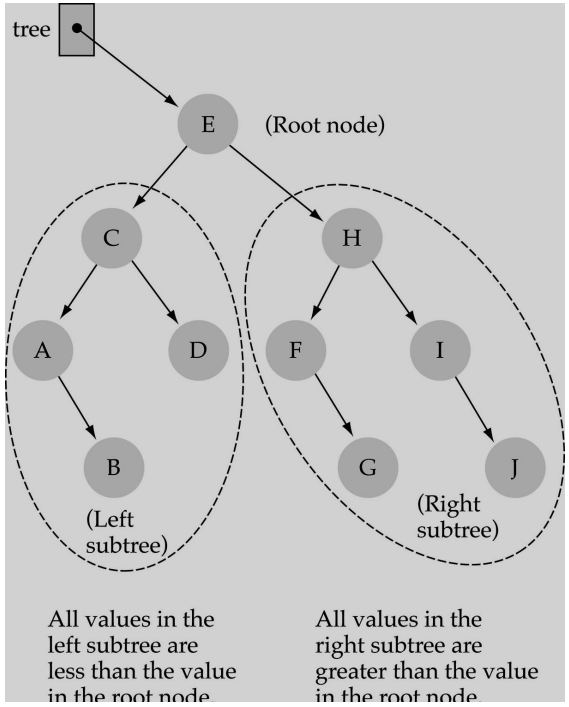
Examples :



In other words, if a key y is in the left subtree of key x then $y.key \leq x.key$ and if y is in the right subtree of key x then $y.key \geq x.key$.

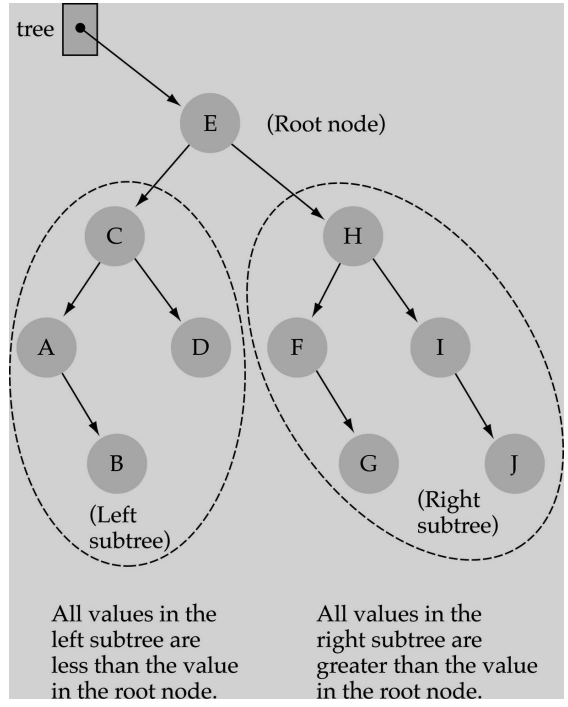
Binary search tree property

- The value stored at the root is greater than the value stored at its left child and less than the value stored at its right child



Binary search tree property

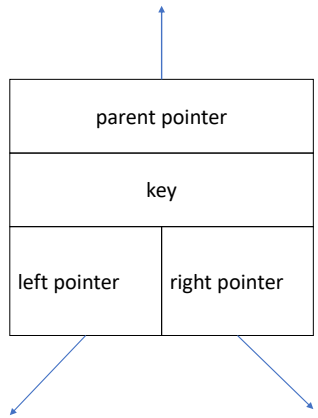
- Same property for the subtrees
- The value stored at the root of a subtree is greater than the value stored at its left subtree and less than the value stored at its right subtree



Nodes declaration in C

A node in a binary search tree is declared as follow

```
typedef struct{
    int key;
 struct node* parent;
    struct node* left;
    struct node* right;
}node;
```

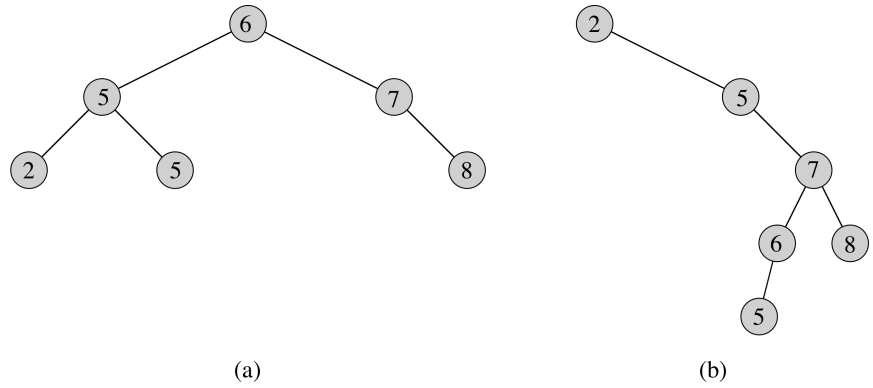


Inorder tree traversal

Inorder tree traversal :

- Visits elements in sorted (increasing) order

InorderTreeWalk(x)
  if $x \neq NIL$
    InorderTreeWalk(x.left) ;
    print(x.key) ;
    InorderTreeWalk(x.right) ;

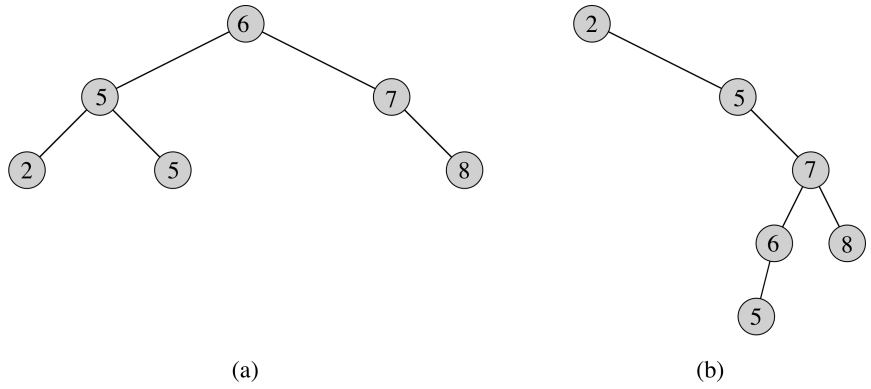


Preorder tree traversal

Preorder tree traversal :

- Visits root before left and right subtrees are visited

PreorderTreeWalk(x)
if $x \neq NIL$
  print(x.key) ;
  PreorderTreeWalk(x.left) ;
  PreorderTreeWalk(x.right) ;

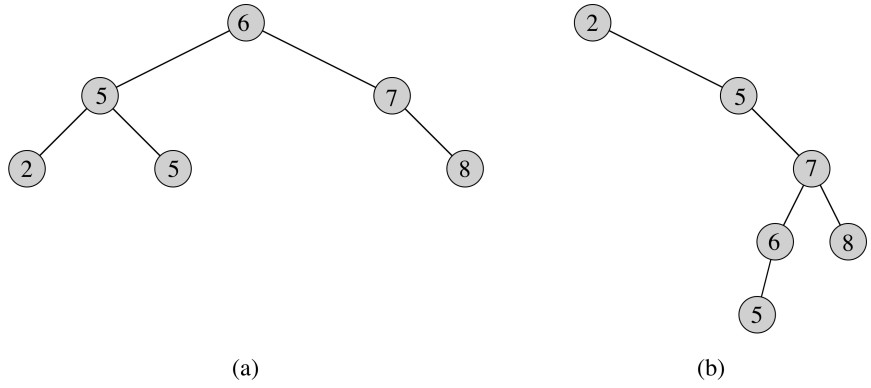


Postorder tree traversal

Postorder tree traversal :

- Visits root after visiting left and right subtrees

```
PostorderTreeWalk(x)
if x ≠ NIL
    PostorderTreeWalk(x.left) ;
    PostorderTreeWalk(x.right) ;
    print(x.key) ;
```



Cost of tree traversals

Theorem : Tree traversal, Inorder, Postorder and Preorder cost $\Theta(n)$ where n is the number of nodes in the tree.

Proof : $T(n)$ is the time for tree traversal called on the root of an n-node subtree. Since each traversal visits the n nodes, we have $T(n) \in \Omega(n)$. We need to show that $T(n) \in O(n)$

Traversal of an empty subtree, for the test $x \neq NIL$, i.e. $T(0)$, cost c

For $n > 0$, suppose a tree traversal is called on a node x where the left subtree has k nodes and the right subtree has $n - k - 1$ nodes

The time to perform a tree traversal from x is bounded by $T(n) \leq T(k) + T(n - k - 1) + d$ where d represents the cost of printing key x

Cost of tree traversals

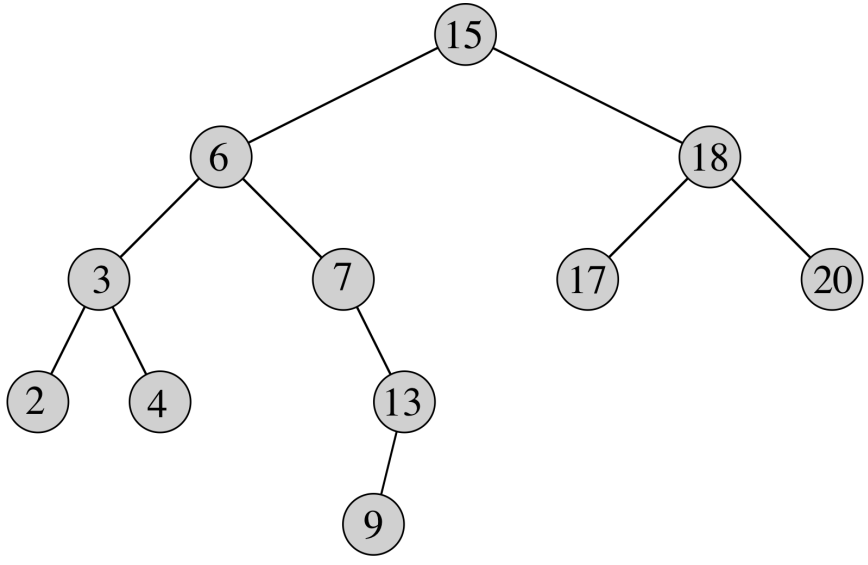
For $n > 0$, $T(n) \leq T(k) + T(n-k-1) + d$

$T(n) \in O(n)$. Proof by substitution, we guess $T(n) \leq (c+d)n + c$ as for each node the test $x \neq NIL$ is executed as well as printing the key x, and there is n nodes. So $c + d$ is the constant amount time for each call + some constant c for empty subtrees.

$$\begin{aligned} T(n) &\leq T(k) + T(n-k-1) + d \\ &= ((c+d)k + c) + ((c+d)(n-k-1) + c) + d \\ &= (c+d)n + c - (c+d) + c + d \\ &= (c+d)n + c \end{aligned}$$

Search for a key k

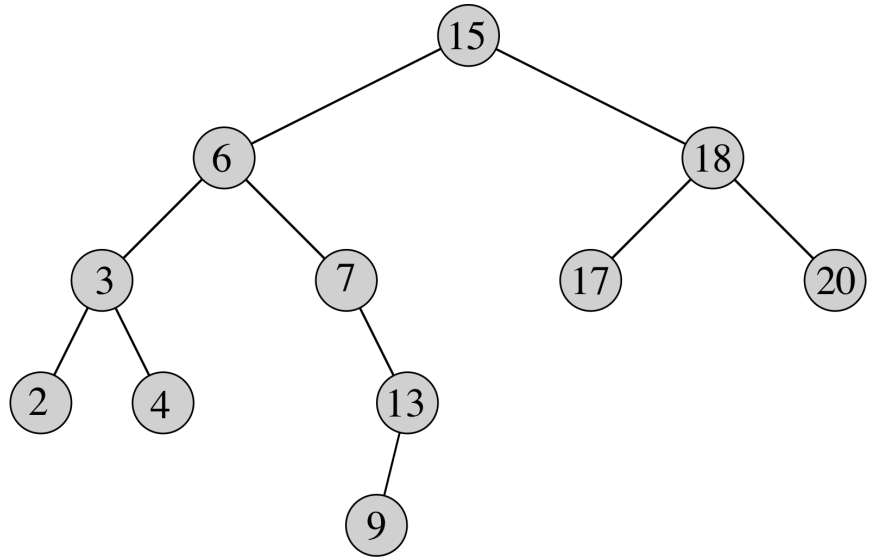
1. Start at the root
2. Compare the key k with the key stored at the root
3. If the values are equal, then item found ; otherwise, if it is a leaf node, then not found
4. If key k less than the value stored at the root, then search the left subtree
5. If key k greater/equal than the value stored at the root, then search the right subtree
6. Back to 2



Operations on BSTs : Recursive Search

Search for a key k at node x (x is a pointer)

```
TreeSearch(x, k)
  if (x = NULL or k = x.key)
    return x;
  if (k < x.key)
    return TreeSearch(x.left, k);
  else
    return TreeSearch(x.right, k);
```



Cost $\Theta(h)$ (where h is the height of the tree) since search is performed along one path in the tree

Operations on BSTs : Iterative Search

Given a key k and a pointer x to a node, the following iterative procedure returns an element with that key or NULL :

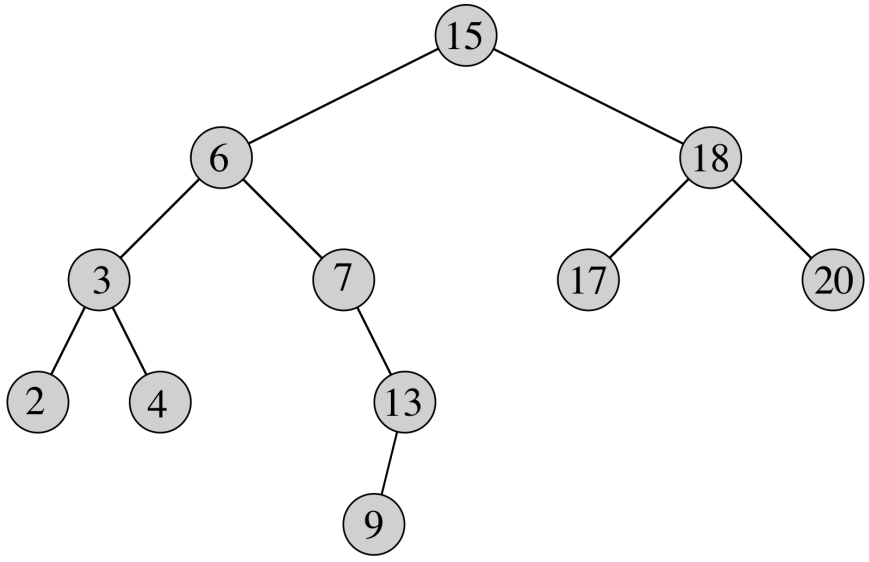
```
TreeSearch(x, k)
  while (x != NULL and k != x.key)
    if (k < x.key)
      x = x.left ;
    else
      x = x.right ;
  return x ;
```

Same asymptotic time complexity $\Theta(h)$, but faster in practice.

Min/max values

Get the key with the minimum or the maximum value.

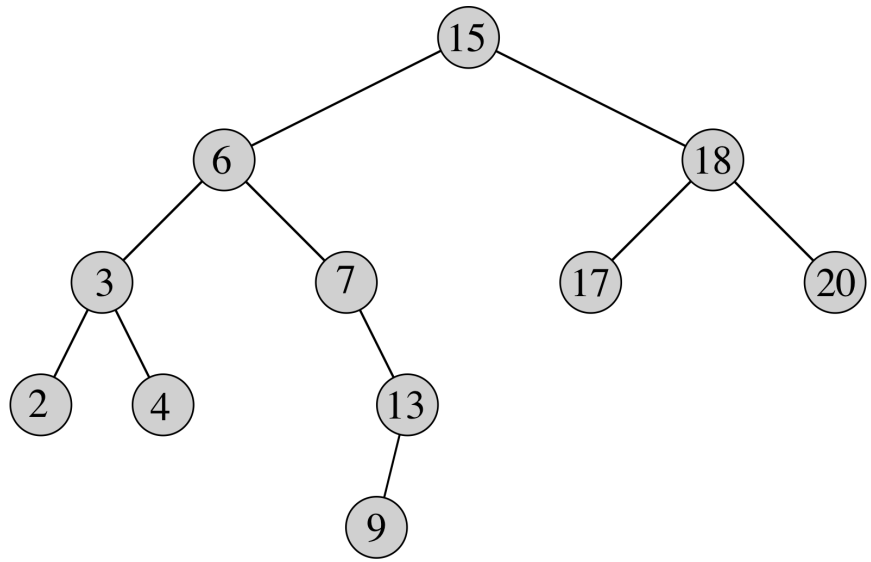
- **Minimum :** return the leaf node of the left most branch in the tree
- **Maximum :** return the lead node of the right most branch in the tree



Min/max values

Get the key with the minimum or the maximum value. These algorithms return a pointer to the node with the min or max value

```
Tree-Minimum(x)
   while x.left ≠ NULL
      x = x.left
   return x
Tree-Maximum(x)
   while x.right ≠ NULL
      x = x.right
   return x
```



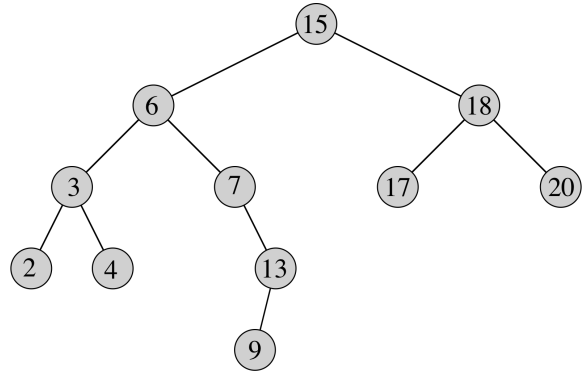
$\Theta(h)$

Successor operation

Given a node x, get its successor node in the sorted order of the keys.

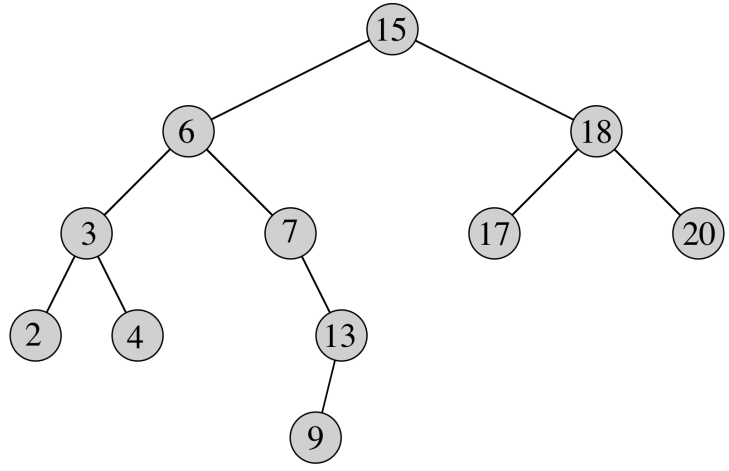
Successor :

- x has a right subtree : successor is minimum node in right subtree
- x has no right subtree : successor is first ancestor of x whose left child is also ancestor of x
  - Intuition : As long as you move to the left up the tree, you're visiting smaller nodes.



Successor Operation

```
Tree-Successor(x)
  if x.right ≠ NULL
     return Tree-Minimum(x.right)
  y = x.p
  while y ≠ NIL and x == y.right
     x = y ; y = y.p
  return y
```

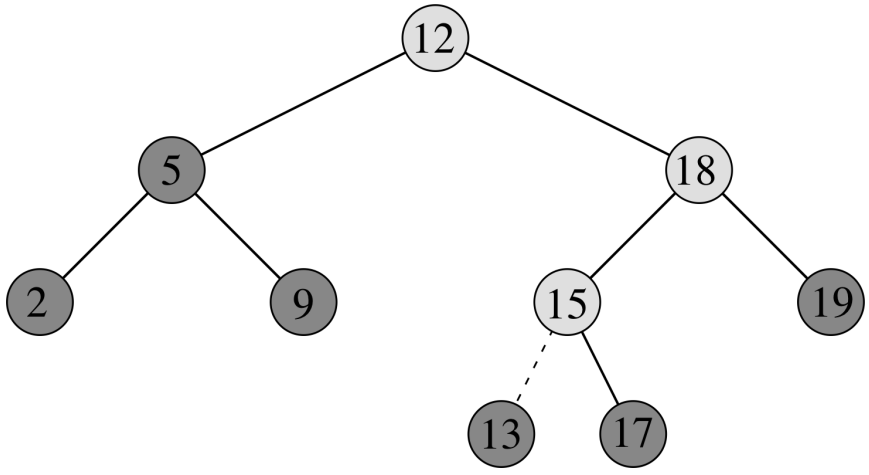


$\Theta(h)$. Tree-predecessor symmetric to Tree-successor

Operations of BSTs : Insert

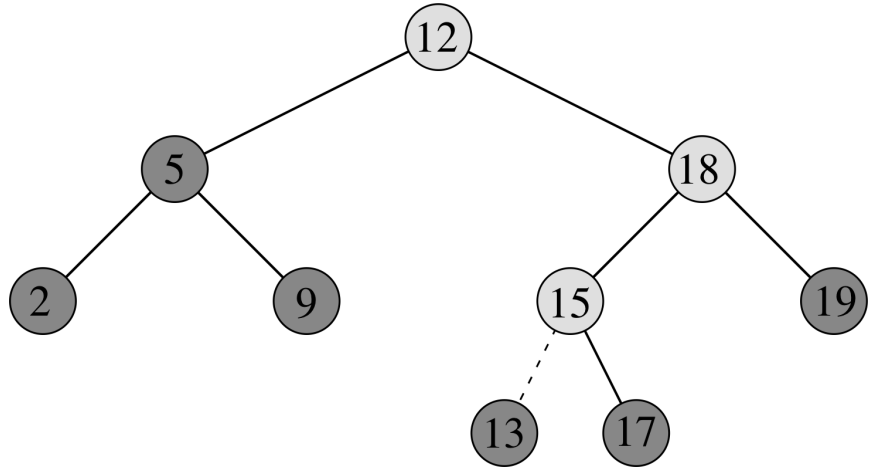
Given a node z with key k

- Run the seach procedure comparing k with each node on the search path
- When on the search path we find a next pointer which is NULL, that is where the node with key k should be inserted



Operations of BSTs : Insert

```
Tree-Insert(T, z)
1    y = NIL
2    x = T.root
3    while x ≠ NIL
4        y = x
5        if z.key < x.key
6            x = x.left
7        else x = x.right
8    z.p = y
9    if y == NIL /* Tree was empty */
10       T.root = z
11   elseif z.key < y.key
12       y.left = z
13   else y.right = z
```



BST Search/Insert : Running Time

The running time of TreeSearch() or Tree-Insert() is $O(h)$, where $h =$ height of tree

The height of a binary search tree in the worst case is $h = O(n)$ when tree is just a linear string of left or right children

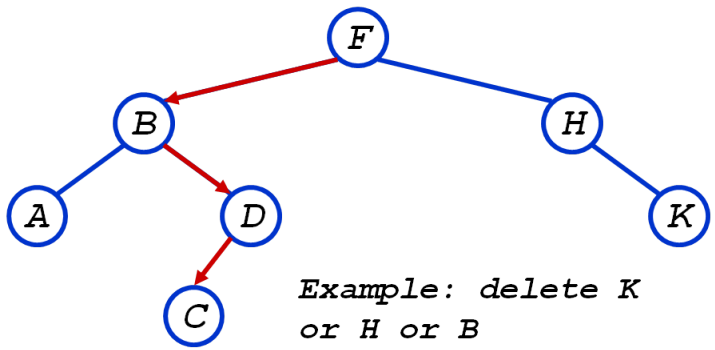
This worst case happens if keys are selected to be inserted in the tree in increasing or decreasing order

- We kept all analysis in terms of h so far for now
- We can maintain $h = O(\lg n)$ in a similar way as for quick sort by randomly picking among the keys the next one that is inserted in the tree

Operations of BSTs : Delete

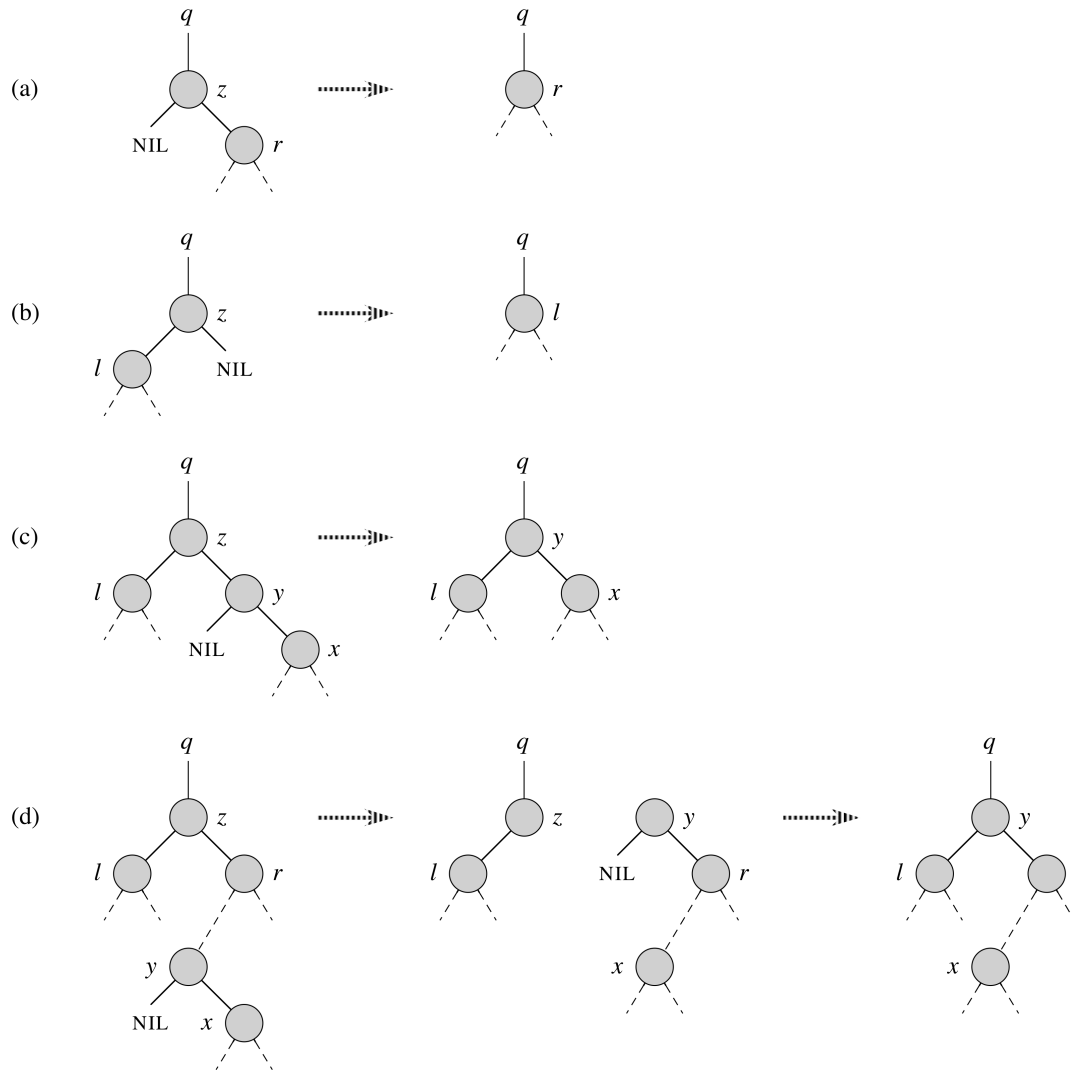
The operation to delete a key z has 3 cases :

1. z has no children : Remove z
2. z has one child : Replace z by his child
3. z has two children :
   - Swap z with successor
   - Perform case 1 or 2 to delete it



BST delete : case 1 and 2

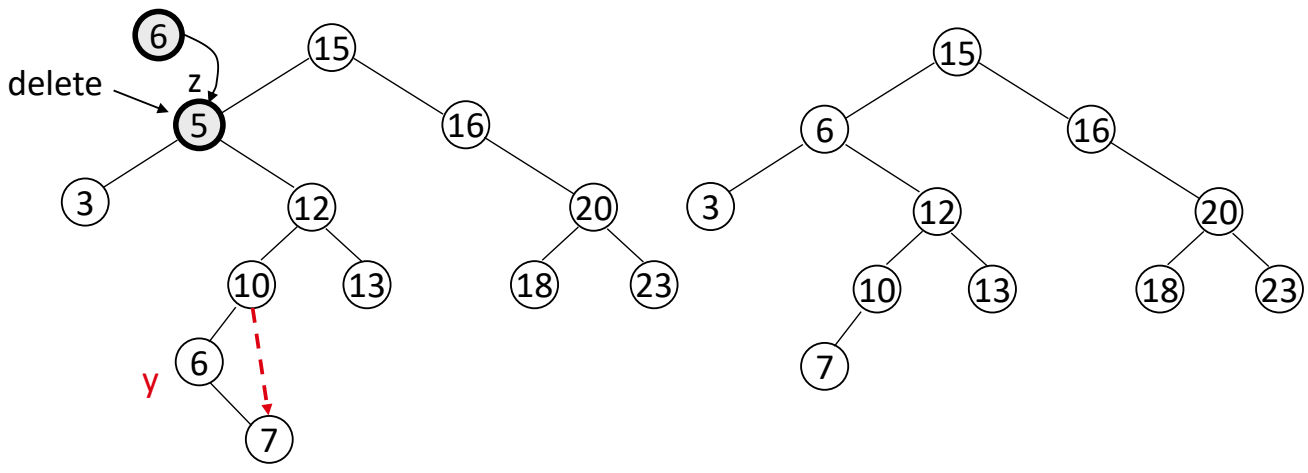
```
Tree-Delete(T, z)
  if z.left == NIL
     r.p = q ; q.right or q.left = x
  elseif z.right == NIL
     l.p = q ; q.right or q.left = l
  else
     y = Tree-minimum(z.right)
```



BST delete case 3

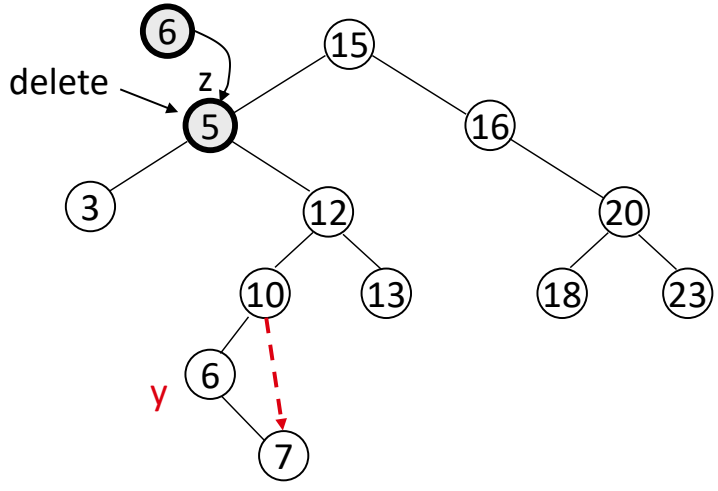
Case 3 : z has two children

- z's successor (y) is the minimum node in z's right subtree
- y has either no children or one right child (but no left child)
- Delete y from the tree (via Case 1 or 2)
- Replace z's key and satellite data with y's.



BST delete case 3

```
Tree-Delete(T, z)
  if z.left == NIL
     r.p = q ; q.right or q.left = x
  elseif z.right == NIL
     l.p = q ; q.right or q.left = l
  else
     y = Tree-minimum(z.right)
     if y.p ≠ z
        y.right.p = y.p ; y.p.left = y.right
        y.right = z.right
        y.right.p = y
     y.p = z.p
     z.p.left = y
     y.left = z.left
     y.left.p = y
```



Sorting with BST

Informal code for sorting array A of length n :

```
BSTSort(A)
   for i=1 to n
      Tree-Insert(A[i]) ;
   InorderTreeWalk(root) ;
```

The "for" loop executes n iterations, each iteration i calls Tree-Insert(A[i]) which is in $O(\log n)$ in best and average cases. Total cost is $n \log n$.

The last part, InorderTreeWalk(root), runs in $O(n)$ as already proved. Therefore $O(n) + O(n \log n) = O(n \log n)$

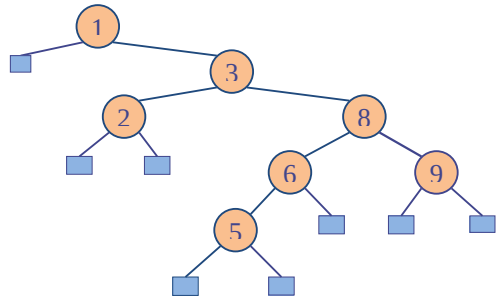
In the worst case we have Tree-Insert(A[i]) running in $O(n)$ (when tree is just a linear string of left or right children). Therefore $n \times O(n) = O(n^2)$

Exercises

1. Draw a binary search tree for the nodes 11, 8, 14, 5, 9, 13, 16, 3, 6, 10, 17, 1
2. Draw the binary search tree from inserting the following sequence of keys : 49 75 92 24 107 26 112 101 19 2 11 25 59 16 28 95
3. Draw a binary search tree containing keys 8, 27, 13, 15, 32, 20, 12, 50, 29, 11, inserted in this order. Then, add keys 14, 18, 30, 31, in this order, and again draw the tree. Then delete keys 29 and 27, in this order, and again draw the tree
4. Beginning with an empty binary search tree, what binary search tree is formed when you insert the following values in the order given ?
   - 4.1 W, T, N, J, E, B, A
   - 4.2 W, T, N, A, B, E, J
   - 4.3 A, B, W, J, N, T, E

Exercise 5

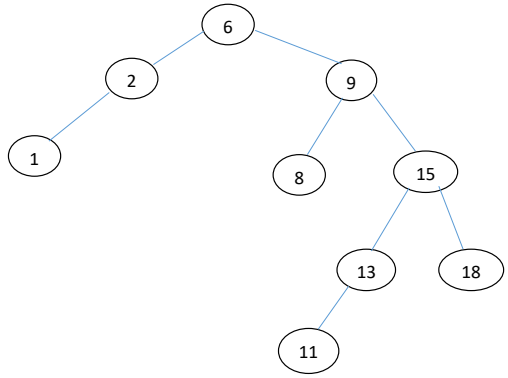
Considering the BST below



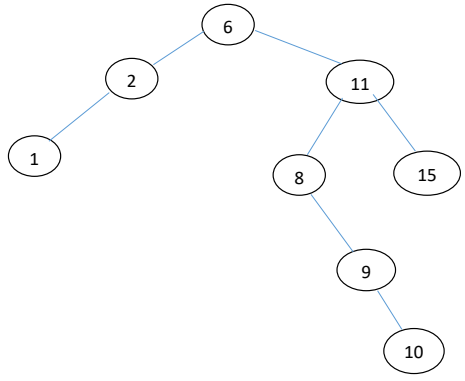
5.1 Show the steps to insert two nodes with both keys as 7.

5.2 Show the steps to delete the node with key=8 (after inserting two nodes with key=7).

Exercises 6 and 7



Considering the BST above, show the steps to delete the node with key=9.



Considering the BST above, show the steps to delete the node with key=6.

Exercises

8. Suppose that we have numbers between 1 and 1000 in a binary search tree, and we want to search for the number 363. Which of the following sequences could not be the sequence of nodes examined ?

   8.1 2, 252, 401, 398, 330, 344, 397, 363.
   8.2 924, 220, 911, 244, 898, 258, 362, 363.
   8.3 925, 202, 911, 240, 912, 245, 363.
   8.4 2, 399, 387, 219, 266, 382, 381, 278, 363.
   8.5 935, 278, 347, 621, 299, 392, 358, 363.

9. Draw the binary search tree obtained when the keys 1, 2, 3, 4, 5, 6, 7 are inserted in the given order into an initially empty tree. What is the problem of the tree you get ? Why is it a problem ? How could you modify the insertion algorithm to solve this problem ?

10. How would you represent a binary search tree using an array ? Give the array representation the following keys inserted consecutively 8, 27, 13, 15, 32, 20 in a binary search tree. Draw the binary search tree corresponding to the array representation below

0	1	2	3	4	5	6	7	8	9	10	11	12
23	2	35	1	13	29	80			11	14	27	

Exercises

11. Write a (recursive or iterative) tree-search algorithm called $Ternary - Tree - Search(x, k)$ that takes the root of a ternary tree and returns the node containing key k. A ternary tree is conceptually identical to a binary tree, except that each node x has two keys, $x.key1$ and $x.key2$, and three links to child nodes, $x.left$, $x.center$, and $x.right$, such that the left, center, and right subtrees contains keys that are, respectively, less than $x.key1$, between $x.key1$ and $x.key2$, and greater than $x.key2$. Assume there are no duplicate keys.
12. You have a set of integers that are stored in a max-heap and in a binary search tree. How will you print in decreasing order the integers in these two data structures ? Is one of these two structures better suited in terms of computational cost to perform this printing of the integers in decreasing order ?

13. Delete node 65 in the graph below (you can find this graph on https ://www.geeksforgeeks.org/binary-search-tree-set-2-delete).